Apuntes – MVVM y Repository

Desarrollo de Aplicaciones I · UADE · Guía de Ejercicios: MVVM y Repository (v1.0)

Proyecto: `MvvmLab` · Felipe Forte

Ejercicio 1 — ¿Quién está haciendo demasiado?

Consigna: tomar una pantalla donde todo está mezclado, marcar dónde se dibuja la interfaz, dónde se validan los datos, dónde se modifica la lista y dónde se guarda, y ubicar cada responsabilidad en *View*, *ViewModel* o *Model/Repository*. Todavía **no** se cambia la implementación.

Conceptos base

MVVM = separar en 3 responsabilidades

- View (Compose) **dibuja** el estado y **avisa** lo que hace el usuario (clicks, texto). No decide nada.
- ViewModel **recibe** la acción del usuario, **decide** (valida, coordina) y **expone** el estado de pantalla.
- Model / Repository **obtiene y guarda** los datos (memoria, archivo, Room, API). No sabe nada de pantallas.

UI = f(estado): la pantalla es una función del estado.

¿Cómo decido dónde va cada cosa?

El criterio **no** es "¿declara una variable?" — hay variables en las tres capas. Las preguntas útiles son:

1. **¿Esto dibuja algo?** → View. *(Test: si cambio Compose por una web, ¿tengo que reescribir esta línea? Sí → View.)*
2. **¿Es una regla que valdría igual en cualquier interfaz?** → ViewModel.
3. **¿Este dato tiene que sobrevivir si cierro la app?** Sí → dato real → Repository. No, solo importa mientras uso la pantalla → estado de pantalla → ViewModel.

Ejemplo: en WhatsApp, el mensaje que estás escribiendo (borrador) es de la pantalla; los mensajes ya enviados se guardan.

El ViewModel decide QUÉ mostrar. La View decide CÓMO mostrarlo.

"Hay un error: tarea vacía" → ViewModel. "El error va en rojo, debajo del botón" → View.

Código analizado (pantalla “mal hecha a propósito”)

```
@Composable
fun PantallaTareas(context: Context, modifier: Modifier = Modifier) {
    var texto by remember { mutableStateOf("") }                    // (A)
    var tareas by remember { mutableStateOf(listOf<String>()) }     // (B)
    var error by remember { mutableStateOf<String?>(null) }         // (C)

    Column(modifier.padding(16.dp)) {                                // (D)
        Text("Mis tareas (${tareas.size})")
        OutlinedTextField(value = texto, onValueChange = { texto = it })

        Button(onClick = {
            if (texto.isBlank()) {                                   // (E)
                error = "La tarea no puede estar vacía"
            } else if (tareas.contains(texto)) {                     // (F)
                error = "Esa tarea ya existe"
            } else {
                tareas = tareas + texto                              // (G)
                File(context.filesDir, "tareas.txt")                 // (H)
                    .writeText(tareas.joinToString("\n"))
                texto = ""                                           // (I)
                error = null
            }
        }) { Text("Agregar") }

        error?.let { Text(it, color = Color.Red) }                   // (J)
        LazyColumn { items(tareas) { Text(it) } }                    // (K)
    }
}
```

Resultado: tabla de responsabilidades (entregable)

Letra	Código	Qué hace	Dónde va
(A)	`var texto`	Borrador que escribe el usuario; no sobrevive al cierre	ViewModel
(B)	`var tareas`	Lista real de tareas; tiene que sobrevivir al cierre	Repository
(C)	`var error`	Mensaje de error actual (dato del estado de pantalla)	ViewModel
(D)	`Column`, `Text`, `OutlinedTextField`	Dibuja la pantalla	View
(E)	`texto.isBlank()`	Regla: no se agregan tareas vacías	ViewModel
(F)	`tareas.contains(texto)`	Regla: no se agregan tareas repetidas	ViewModel
(G)	`tareas = tareas + texto`	Agrega la tarea a la lista (el dueño del dato la modifica)	Repository
(H)	`File(...).writeText(...)`	Guarda las tareas en un archivo	Repository
(I)	`texto = ""`	Limpia el campo solo si la tarea se agregó bien (decisión sobre un dato del ViewModel)	ViewModel
(J)	`error?.let { Text(it, Color.Red) }`	Muestra el error en rojo (el error es del VM; dibujarlo es de la View)	View
(K)	`LazyColumn`	Muestra la lista en pantalla	View

Errores frecuentes (los que cometí y corregí)

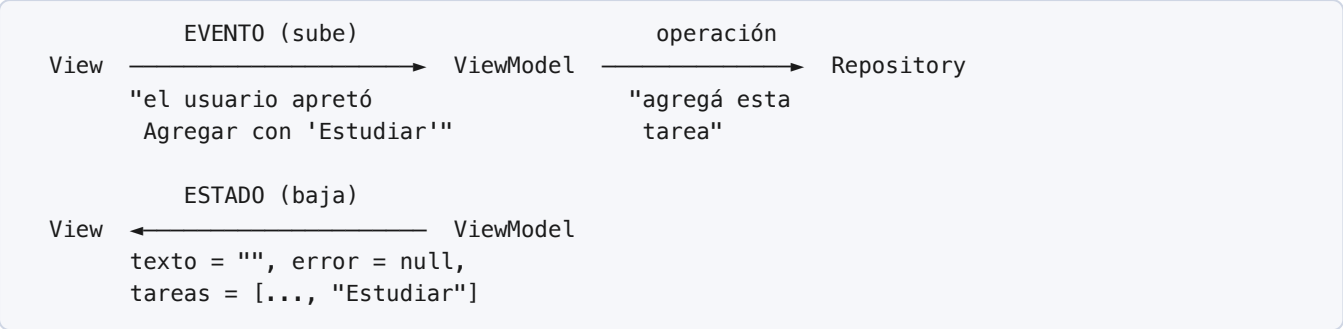
- Pensar que (A), (B) y (C) van al Repository "porque declaran variables". Solo (B) es un dato que debe persistir.
- Poner (I) en la View: `texto` es del ViewModel, y quien es dueño de un dato es quien lo modifica.
- Poner (J) en el ViewModel: el *error* es del ViewModel, pero *mostrarlo* con `Text` y `Color.Red` es de la View.

La cadena completa de "Agregar"

1. View el usuario aprieta *Agregar* → avisa al ViewModel.
2. ViewModel aplica las reglas (E) y (F). Si pasan → le pide al Repository que agregue; limpia el campo (I).
3. Repository agrega la tarea (G) y la guarda en el archivo (H).

Repository = **dueño del dato** · ViewModel = **intermediario** (lo pide y lo prepara) · View = **solo lo muestra**. La lista "verdadera" vive en un solo lugar: **fuente de verdad única**.

Desafío: las dos flechas (UDF – flujo unidireccional de datos)



> **Evento vs. estado**
>
> - **Evento** = algo que *pasó* (un click, una tecla). Es puntual.
> - **Estado** = cómo *está* la pantalla ahora. Se puede consultar en cualquier momento.
> - **UDF:** los eventos suben y el estado baja. La View nunca modifica el estado por su cuenta: solo avisa y redibuja.

Preguntas de la guía

1. ¿Qué trabajo tendría que seguir funcionando aunque mañana cambiemos Compose por otra interfaz?
Las **reglas** (E, F: no vacía, no repetida) y el **manejo de datos** (G, H: agregar y guardar). Es decir, todo lo que va en ViewModel y Repository.

2. ¿Qué datos existen sólo mientras se está escribiendo un formulario?
`texto` (el borrador) y `error` (el mensaje actual). Son estado de pantalla y no se guardan en el archivo.

> **Señal de alarma: `Context` en un composable**
>
> `PantallaTareas` recibe un `Context` solo para poder escribir el archivo (H). La guía advierte no aceptar soluciones que "llevan Context a todos los composables" o "abren archivos desde un botón". Es una pista de que el acceso a archivos está en el lugar equivocado: debería estar en el Repository.

Kotlin / Compose que apareció en el camino

Elemento	Qué es
`Unresolved reference: X`	Kotlin no sabe qué es X: falta el `import`, está mal escrito o la función/clase no existe.
`var x by remember { mutableStateOf(...) }`	Estado observable que sobrevive a la recomposición. El `by` necesita los imports `getValue` / `setValue`.
`LocalContext.current`	Forma de obtener el `Context` de Android dentro de un composable.
`modifier: Modifier = Modifier`	Convención: permite que quien llama le pase padding, tamaño, etc. (por ejemplo el `innerPadding` del Scaffold).
`@Preview` / `@Composable`	Anotaciones: "etiquetas" que siempre se aplican a una función. Solas, sin función debajo, no compilan.

Ejercicio 2 — Una pantalla no es su estado

Consigna: pantalla de notas con título, contenido y botón Guardar. Definir una `data class` de estado de pantalla con el contenido actual, el error de validación y si hay una operación en curso. Simular el guardado. Identificar eventos separados para editar el título, editar el texto y pulsar Guardar. Todavía **sin ViewModel**.
Archivo: `Ej02PantallaNota.kt` · Rama: `ej02-estado-pantalla`

> **Estado de pantalla (UiState)**
> Una sola "foto" de todo lo que la pantalla necesita para dibujarse. En el Ej 1 había 3 variables sueltas; ahora todo está junto en un objeto. **Si sabés el estado, sabés exactamente qué se ve.**

El estado

```
data class Nota(val titulo: String, val contenido: String)   // una nota YA guardada

data class NotaUiState(
    val titulo: String = "",          // lo escrito en el campo título
    val contenido: String = "",       // lo escrito en el campo contenido
    val error: String? = null,        // error de validación (null = sin error)
    val guardando: Boolean = false,   // operación en curso
    val guardada: Nota? = null        // última nota guardada (null = ninguna)
) {
    val hayCambiosSinGuardar: Boolean get() = ...   // propiedad CALCULADA
}
```

Elemento Kotlin	Por qué se usa
`data class`	Clase pensada para representar datos: genera `equals`, `toString` y `copy`.
Campos `val` + `copy(...)`	El estado es inmutable: para cambiarlo se crea una copia nueva con los campos modificados. Cada cambio es un estado nuevo y Compose detecta que tiene que redibujar.
`String?` / `Nota?`	Tipo anulable: el error o la nota guardada pueden no existir (`null`).
Valores por defecto (`= ""`)	`NotaUiState()` ya es el estado inicial: pantalla vacía.
Propiedad calculada (`get() = ...`)	Se deduce de otros campos en vez de guardarse: no hay información duplicada que pueda quedar desincronizada.

Desafío: tabla evento → cambio de estado (antes de codificar)

Evento	Cambio de estado esperado
Edita el **título**	`titulo = nuevo`, `error = null` (el error deja de tener sentido al escribir)
Edita el **contenido**	`contenido = nuevo`
Pulsa **Guardar** con título vacío	`error = "El título no puede estar vacío"`
Pulsa **Guardar** con título válido	`guardando = true`, `error = null`
Pulsa **Guardar** mientras ya guarda	Sin cambios: se ignora (y el botón está deshabilitado)
Termina el guardado simulado	`guardando = false`, `guardada = Nota(titulo, contenido)`

Cómo quedó organizado el código

Pieza	Rol
`Nota`	Modelo: una nota ya guardada.
`NotaUiState`	Estado de pantalla: **qué** se ve.
`alEditarTitulo`, `alEditarContenido`, `alPulsarGuardar`, `alTerminarGuardado`	Funciones Kotlin **puras**: reciben el estado actual y devuelven el nuevo. No conocen Compose. En el Ej 3 se mudan al ViewModel casi sin cambios.
`PantallaNotaConEstado`	Dueño del estado (provisorio, hasta el Ej 3) y del guardado simulado con `delay`.
`PantallaNota`	View **sin estado**: dibuja el `NotaUiState` que recibe y avisa los eventos por callbacks.
4 `@Preview`	Vacía · Error · Guardando · Guardada con cambios sin guardar.

State hoisting (elevar el estado)

```
@Composable
fun PantallaNota(
    estado: NotaUiState,                  // el estado BAJA
    onTituloChange: (String) -> Unit,     // los eventos SUBEN
    onContenidoChange: (String) -> Unit,
    onGuardar: () -> Unit,
    modifier: Modifier = Modifier
)
```

La View no guarda ni decide nada: recibe un valor y funciones (callbacks) para avisar. **Ventaja concreta:** como no tiene estado propio, se puede previsualizar en *cualquier* situación pasándole un `NotaUiState` armado a mano. Es el mismo UDF del Ej 1, ahora en código.

Lambdas y tipos función

`(String) -> Unit` = "una función que recibe un String y no devuelve nada". Se pasa como parámetro: `onTituloChange = { estado = alEditarTitulo(estado, it) }`. Dentro de la lambda, `it` es el único parámetro (el texto nuevo).

Guardado simulado: corrutina

`rememberCoroutineScope()` + `scope.launch { delay(1500); ... }` espera 1,5 s *sin congelar la pantalla*. Mientras tanto `guardando = true` → el botón muestra "Guardando..." y queda deshabilitado (`enabled = !estado.guardando`), lo que evita la doble pulsación. En el Ej 3 esto pasa a `viewModelScope`.

Preguntas de la guía

1. ¿El texto ingresado es lo mismo que la nota ya guardada?
No. `titulo` / `contenido` son lo que el usuario **está escribiendo** (cambia en cada tecla); `guardada` es la **última versión confirmada**. Por eso son campos distintos, y comparar ambos es lo que permite mostrar "Hay cambios sin guardar".

2. ¿Conviene mostrar el mensaje de error como parte del estado?
Sí. Si el error está en el estado, la pantalla se reconstruye completa a partir de `NotaUiState` (UI = f(estado)): sobrevive a la recomposición, se puede previsualizar y, con ViewModel (Ej 3), también a la rotación. Un mensaje "suelto" disparado desde el botón (un Toast) no se puede consultar ni recuperar.

Criterio de finalización

✅ Eventos reconocibles: `onTituloChange`, `onContenidoChange`, `onGuardar`.
✅ La pantalla se representa a partir de un único estado (`NotaUiState`): lo prueban las 4 previews, que dibujan cualquier situación sin ejecutar la app.